

# Full Stack Developer Internship

## One-Day Task: Role-Based E-Commerce Platform

**Duration:** 7.5 - 8.5 hours (1 working day; beginners may take 2 days)

---

### 1. Overview

Build one complete e-commerce web application in a single day that proves the intern can independently handle: database design, role-based authentication, CRUD operations, image upload, payment integration, Git workflow, and live deployment.

**Note on tech choices:** The intern is free to choose their own frontend framework, backend framework, and database (SQL or NoSQL). Only the following are fixed requirements: Cloudinary for image uploads, Razorpay for payments, and deployment to Render (backend) + Vercel (frontend).

---

### 2. Roles & Permissions

| Role         | Permissions                                                                                                     |
|--------------|-----------------------------------------------------------------------------------------------------------------|
| Admin        | Full control — manage all products, manage users & their roles, view/update all orders, view basic sales stats. |
| Sales Person | Add, edit, delete only their own products; view orders containing their products.                               |
| User         | Browse, search, filter products; manage wishlist and cart; view their own order history.                        |

**Why it matters:** The real test is enforcing these permissions on the backend — a role check that only hides a button on the frontend doesn't count.

---

### 3. Task Breakdown

#### Task 1 — Project Setup & GitHub Repo (Duration: 20 min)

- Create one repository with a clear frontend/backend folder structure.
- Add `.gitignore` (exclude dependency folders and env files) and a `.env.example` listing every required key, without real values.
- Push the first commit as a baseline.

#### Task 2 — Database Design (Duration: 30 min)

- Design entities for: Users (with a role field), Products (with an owner/seller reference), Cart, Wishlist, Orders.
- Choice of database (SQL/NoSQL) is up to the intern — the goal is a clean, sensible schema.

### **Task 3 — Authentication & Role-Based Access Control (Duration: 60 min)**

- Build register/login with secure password storage (never plain text — use hashing, e.g. bcrypt).
- Issue a session/token on login.
- Add middleware/guards that restrict each route by role, so a restricted action returns an error for a non-admin — not just a hidden UI element.

### **Task 4 — Product CRUD + Cloudinary Image Upload (Duration: 75 min)**

- Public endpoint to list/search/filter products (category, price range, keyword).
- Protected create/update/delete: Admin can manage any product; Sales Person can manage only their own.
- Product images must be uploaded directly to Cloudinary — only the returned image URL is stored in the database, never the raw file on the server.

### **Task 5 — Product Listing, Search & Filters (Frontend) (Duration: 60 min)**

- Responsive product grid with image, name, price, category.
- Search bar and filter controls that query the backend.
- Navigation adapts by role — "Add Product" is visible only to Admin and Sales Person.

### **Task 6 — Wishlist & Cart (Duration: 45 min)**

- Add/remove wishlist items.
- Add to cart with adjustable quantity; cart icon shows item count.

### **Task 7 — Razorpay Checkout (Test Mode) (Duration: 75 min)**

- Backend: create a Razorpay order, then verify the payment signature after checkout (this step prevents fake "success" callbacks).
- Frontend: trigger Razorpay checkout, and on verified success, save the order record and clear the cart.
- Use Razorpay's official test credentials — no real transactions.

### **Task 8 — Order History & Role Dashboards (Duration: 45 min)**

- **User:** view their own past orders.
- **Sales Person:** view orders containing their products.
- **Admin:** view all orders plus a simple stats summary (total sales, total orders, etc.).

### **Task 9 — Git Workflow Practice (Duration: ~15 min, ongoing throughout the day)**

- Commit after each completed milestone (8-10+ commits) with clear, descriptive messages — not one giant commit at the end.
- Create at least one feature branch and merge it into main via a Pull Request.

### **Task 10 — Deployment (Duration: 45 min)**

- **Backend → Render:** connect the GitHub repo, configure build/start commands, add all secrets as environment variables (never hardcoded).

- **Frontend → Vercel:** import the repo, point it at the live Render backend URL.
- Verify the full flow works live: login → browse → cart → payment → order appears.

### Task 11 — README & Submission (Duration: 20 min)

Include the following in your README:

- Stack used (Frontend, Backend, Database)
- Setup & installation steps
- Environment variables required (`.env.example`)
- Test login credentials for each role (Admin, Sales Person, User)
- 2-3 screenshots of the application
- Live Frontend & Backend URLs
- **Feature Completion Summary** explaining what has been implemented

For each feature, briefly mention:

- What you implemented
- How it works
- Any assumptions or limitations (if applicable)

### Example — Feature Completion Summary:

| Feature           | Implementation Summary                                                                                          |
|-------------------|-----------------------------------------------------------------------------------------------------------------|
| Authentication    | JWT-based authentication with bcrypt password hashing.                                                          |
| Role-Based Access | Admin has full access, Sales Person can manage only their own products, Users can browse and purchase products. |
| Product CRUD      | Complete CRUD with ownership validation and Cloudinary image uploads.                                           |
| Wishlist          | Users can add/remove products from their wishlist.                                                              |
| Cart              | Quantity update, remove item, and cart count implemented.                                                       |
| Razorpay          | Test mode payment with backend signature verification before creating orders.                                   |
| Deployment        | Backend deployed on Render and Frontend deployed on Vercel.                                                     |

### Example — Role Credentials:

| Role         | Email                                                    | Password |
|--------------|----------------------------------------------------------|----------|
| Admin        | <a href="mailto:admin@example.com">admin@example.com</a> | *****    |
| Sales Person | <a href="mailto:sales@example.com">sales@example.com</a> | *****    |
| User         | <a href="mailto:user@example.com">user@example.com</a>   | *****    |
|              |                                                          |          |

## 4. Deliverables

The candidate must submit:

- **GitHub Repository Link** (with clean commit history and one merged Pull Request)
- **Live Backend URL** (Render)
- **Live Frontend URL** (Vercel)
- **README** containing:
  - Setup instructions
  - Environment variables (`.env.example`)
  - Test credentials for all roles
  - Screenshots
  - Feature Completion Summary
- **Implementation Report (Mandatory)** — should briefly describe:
  - What features were completed
  - How each feature was implemented
  - Any challenges faced and how they were solved
  - Any pending features or known limitations
  - Test credentials for all roles

## 5. Evaluation Criteria

- Role restrictions enforced on the backend, not just the UI
- Product CRUD and Cloudinary upload genuinely working
- Cart, wishlist, and filters functional
- Razorpay test payment completes end-to-end and creates a real order
- Git commit history quality and correct PR usage
- Both deployments live and reachable